

P1759R0

Native handle from file streams

Published Proposal, 2019-06-17

This version:

<https://wg21.link/P1759R0>

Author:

[Elias Kosunen isocpp@eliaskosunen.com](mailto:Elias.Kosunen@isocpp@eliaskosunen.com)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LEWGI

Source:

github.com/eliaskosunen/wg21: P1759R0

Abstract

This paper proposes adding functionality to `fstream` and `filebuf` to retrieve the native file handle.

Table of Contents

1 Revision History

1.1 Revision 0

2 Overview

3 Motivation

4 Scope

5 Design Decisions

5.1 Free function and namespace-scope typedef

5.2 Separate `native_handle_type` from `thread` and Networking TS

5.3 Type of `native_file_handle_type`

5.4 Naming

6 Design Alternatives

6.1 Member typedef instead of namespace scope typedef

6.2 Member instead of namespace scope

7 Impact On the Standard and Existing Code

8 Implementation

9 Prior Art

9.1 Discussion

10 Technical Specifications

29.9.* Function template `native_file_handle` [`file.handle`]

11 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial revision.

§ 2. Overview

This paper proposes adding a new free function: `std::native_file_handle`. It would take a standard file stream or stream buffer, and return the native file handle corresponding to that stream.

```

template <typename CharT, typename Traits>
auto native_file_handle(const basic_fstream<CharT, Traits>& stream) noexcept

template <typename CharT, typename Traits>
auto native_file_handle(const basic_ifstream<CharT, Traits>& stream) noexcept

template <typename CharT, typename Traits>
auto native_file_handle(const basic_ofstream<CharT, Traits>& stream) noexcept

template <typename CharT, typename Traits>
auto native_file_handle(const basic_filebuf<CharT, Traits>& stream) noexcept

```

The return type of this function is `std::native_file_handle_type`, which is a typedef to whatever type the platform uses for its file descriptors: `int` on POSIX, `HANDLE` (`void*`) on Windows, and something else on other platforms. This type is a non-owning handle and is to be small, `Regular`, and trivially copyable.

§ 3. Motivation

For some operations, using OS/platform-specific file APIs is necessary. If this is the case, they are unable to use iostreams without reopening the file with the platform-specific APIs.

For example, if someone wanted to query the time a file was last modified on POSIX, they'd use `::fstat`, which takes a file descriptor:

```

int fd = ::open("~/foo.txt", O_RDONLY);
::stat s{};
int err = ::fstat(fd, &s);
std::chrono::sys_seconds last_modified = std::chrono::seconds(s.st_mtime.tv

```

Note: The Filesystem TS introduced the `file_status` structure and `status` function retuning one. This doesn't solve our problem, because `std::filesystem::status` takes a path, not a native file descriptor (using paths is potentially racy), and `std::filesystem::file_status` only contains member functions `type()` and `permissions()`, not one for last time of modification. Extending this structure is out of scope for this proposal.

If the user needs to do a single operation not supported by the standard library, they have to make choice between only using OS APIs, or reopening the file every time necessary, likely forgetting to close the file, or running into buffering or synchronization issues.

```
// Writing the latest modification date to a file
std::chrono::sys_seconds last_modified(int fd) {
    // See above
}

// Today's code
{
    int fd = ::open("~/foo.txt", O_RDONLY); // CreateFile on Windows
    auto lm = last_modified(fd);

    // Using iostreams
    if (use_iostreams) {
        ::close(fd); // CloseFile on Windows
        // Hope the path still points to the file!
        std::ofstream of("~/foo.txt");
        of << std::chrono::format("%c", lm) << '\n';
    }

    // Or using POSIX ::write()
    if (use_posix) {
        // Using ::write() is clunky;
        // skipping error handling for brevity
        auto str = std::chrono::format("%c", lm);
        str.push_back('\n');
        ::write(fd, str.data(), str.size());
        // Remember to close!
        ::close(fd);
    }
}

// This proposal
{
    // No need to use platform-specific APIs to open the file
    std::ofstream of("~/foo.txt");
    auto lm = last_modified(std::native_file_handle(of));
    of << std::chrono::format("%c", lm) << '\n';
    // RAII does ownership handling for us
}
```

The utility of getting a file descriptor (or other native file handle) is not limited to getting the last modification date. Other examples include, but are definitely not limited to:

- file locking (`fcntl()` + `F_SETLK` on POSIX, `LockFile` on Windows)
- getting file status flags (`fcntl()` + `F_GETFL` on POSIX, `GetFileInformationByHandle` on Windows)
- non-blocking IO (`fcntl()` + `O_NONBLOCK/F_SETSIG` on POSIX)

Basically, this paper would make standard file streams interoperable with operating system interfaces, making iostreams more useful in that regard.

Facilities replacing iostreams, although desirable, are not going to be available in the standard in the near future. The author, alongside many others, would thus find this functionality useful. When an iostreams replacement eventually arrives, the overload set can then be extended for easy interop.

§ 4. Scope

This paper does *not* propose constructing a file stream or stream buffer from a native file handle. The author is worried of ownership and implementation issues possibly associated with this design.

```
// Not part of this proposal
// POSIX example
#include <fstream>
#include <fcntl.h>

std::native_file_handle_type fd = ::open(/* ... */);
auto f = std::fstream{fd};
```

This paper also does *not* propose getting the native handle from a C-style file stream `FILE`.

```
// Not part of this proposal
#include <cstdio>

auto fd = std::native_file_handle(stdout); // stdout is of type FILE*
// This would essentially be a wrapper over POSIX fileno or Windows _fileno
```

The author is open to extending the paper to cover one or all of these areas, in this paper or some separate one (whichever is more appropriate), should there be a desire and consensus for doing so.

§ 5. Design Decisions

§ 5.1. Free function and namespace-scope typedef

In this proposal, `native_file_handle` is a namespace-scope free function to avoid needing to add a new `virtual` function, causing an ABI break.

This paper opted for a namespace scope typedef for deliberately introducing inconsistency; `std::thread` has a member function `native_handle`, and since we have a free function, a namespace scope typedef is used to draw attention to the fact that the interface is different.

§ 5.2. Separate `native_handle_type` from `thread` and Networking TS

C++11 thread support library includes functionality for getting a native handle out of a `std::thread`. Several types in there have members `native_handle_type` and `native_handle`. The same case also applies for the Networking TS [N4734]. The author feels like it'd be good design to keep `native_file_handle_type` separate from threads for the sake of type safety, even though they could be the same underlying type.

§ 5.3. Type of `native_file_handle_type`

This paper describes `native_file_handle_type` as a typedef to the native file descriptor type. Alternatives to this could be:

- making it an `enum class`, à la `std::byte`
- making it a standard layout `struct` with a member function/variable returning the actual handle

`std::thread` defines its `native_handle_type` as an implementation-defined typedef, just like this paper does.

§ 5.4. Naming

The names `native_file_handle` and `native_file_handle_type` are subject to bikeshedding. A non-exhaustive list of alternatives:

Function	Type
<code>get_native_file_handle</code>	<code>native_file_handle</code>
<code>native_handle</code>	<code>native_handle_type</code>

Function	Type
<code>get_native_handle</code>	<code>native_handle</code>
<code>get_native_handle</code>	<code>native_handle_type</code>
<code>file_descriptor</code>	<code>file_descriptor_type</code>
<code>get_file_descriptor</code>	<code>file_descriptor</code>
<code>native_file_descriptor</code>	<code>native_file_descriptor_type</code>
<code>get_native_file_descriptor</code>	<code>native_file_descriptor</code>

§ 6. Design Alternatives

§ 6.1. Member typedef instead of namespace scope typedef

It could be a viable alternative to provide `native_file_handle_type` as a member typedef inside `basic_filebuf` and `basic_(i|o)fstream`.

```
template <class CharT, class Traits>
class basic_filebuf {
public:
    using native_handle_type = /* implementation-defined */
    // ...
};
// Same for file streams

template <class CharT, class Traits>
auto native_file_handle(const basic_filebuf<CharT, Traits>& buf) noexcept
    -> typename decltype(buf)::native_handle_type;
// Also overloads for file streams
```

§ 6.2. Member instead of namespace scope

This paper proposes adding a free function and a typedef in namespace scope. Should adding a member function be possible (having it not be `virtual`), there is an alternative design:

```

template <class CharT, class Traits>
class basic_filebuf {
public:
    using native_handle_type = /* implementation-defined */
    // ...
    native_handle_type native_handle() const noexcept;
};
// Same for file streams

```

The author would prefer this, if possible, due to consistency with `std::thread` and the Networking TS.

However, the author is doubtful about implementability and usability of this. See [§5.1 Free function and namespace-scope typedef](#) for rationale.

Should this alternative design be adopted, the names should be changed from `native_file_handle_type` and `native_file_handle` to `native_handle_type` and `native_handle`, respectively (dropping the `file`), again for consistency with `thread`.

§ 7. Impact On the Standard and Existing Code

This proposal is a pure library extension, requiring no changes to the core language. It would cause no existing conforming code to break.

§ 8. Implementation

Implementing this paper should be a relatively trivial task. The only issue is, that a lot of the data is hidden behind a private interface, so modification of the library internals would be required. To go around this, the following reference implementations use an exposition-only function `__get_cfile_handle`, which returns an internal C stdio file handle and could be implemented as a `friend`.

Although all implementations surveyed (libstdc++ and MSVC) use `FILE*` instead of native file descriptors in their `basic_filebuf` implementations, these platforms provide facilities to get a native handle from a `FILE*`; `fileno` on POSIX, and `_fileno` + `__get_osfhandle` on Windows. The following reference implementations use these.

For libstdc++ on Linux:


```

namespace std {
    using native_file_handle_type = int;

    template <class CharT, class Traits>
    native_file_handle_type native_file_handle(const basic_filebuf<CharT, 1
        // _M_file is a protected member variable of basic_filebuf,
        // so using friend __get_cfile_handle instead
        const __basic_file<char>& file = __get_cfile_handle(buf);
        // __basic_file<char> has a member function for this purpose
        return file.fd();
        // ::fileno(file.file()) could also be used
    }

    // Other overloads are trivial with rdbuf()
}

```

For MSVC:

```

namespace std {
    using native_file_handle_type = HANDLE;

    template <class CharT, class Traits>
    native_file_handle_type native_file_handle(const basic_filebuf<CharT, 1
        // _Myfile is a private member of basic_filebuf,
        // so using friend __get_cfile_handle instead
        auto cfile = ::_fileno(__get_cfile_handle(buf));
        return static_cast<HANDLE>(__get_osfhandle(cfile));
    }

    // Other overloads are trivial with rdbuf()
}

```

§ 9. Prior Art

[Boost.IOStreams] provides `file_descriptor`, `file_descriptor_source`, and `file_descriptor_sink`, which, when used in conjunction with `stream_buffer`, are `std::basic_streambuf`s using a file descriptor. These classes can be constructed from a path or a native handle (`int` or `HANDLE`) and can also return it with member function `handle()`.

Niall Douglas's [\[P1031R1\]](#) also defined a structure `native_handle_type` with an extensive interface and a member `union` with an `int` and a `HANDLE`, with a constructor taking either one of these.

§ 9.1. Discussion

There has been some discussion over the years about various things relating to this issue, but as far as the author is aware, no concrete proposal has ever been submitted.

There have been a number of threads on `std-discussion` and `std-proposals`:

[\[std-proposals-native-handle\]](#), [\[std-discussion-fd-io\]](#), [\[std-proposals-native-raw-io\]](#), [\[std-proposals-fd-access\]](#). The last one of these lead to a draft paper, that was never submitted: [\[access-file-descriptors\]](#).

The consensus that the author took from these discussions is, that native handle support for `iostreams` would be very much welcome.

An objection was raised by Billy O'Neal to being able to retrieve a native file handle from a standard file stream:

[This] also would need to mandate that the C++ streams be implemented directly such that there was a 1:1 native handle relationship, which may not be the case. For instance, a valid implementation of C++ `iostreams` would be on top of `cstdio`, which would not have any kind of native handle to expose.

– Billy O'Neal: [\[std-proposals-billy-oneal\]](#)

Every implementation surveyed did implement `basic_filebuf` on top of C `stdio`, but these platforms also provide functionality for getting a file descriptor out of a `FILE*`. On every platform, file I/O is ultimately implemented on top of native APIs, so not providing access to a file descriptor from a `FILE*` would be rather unfortunate. Should such a platform exist, they probably don't have a conforming C++ implementation anyway. See [§8 Implementation](#) for more.

§ 10. Technical Specifications

The proposed wording is likely to be incomplet and/or incorrect.

Add the following into Header `<iosfwd>` synopsis [`iosfwd.syn`]:

```
using native_file_handle_type = /* implementation-defined */;
```

Add the following two paragraphs between § 5 and § 6 of *Overview* [**iostream.forward.overview**]:

The type `native_file_handle_type` serves as a type representing a platform-specific handle to a file. It satisfies the requirements of `Regular` and is trivially copyable.

[*Note*: For operating systems based on POSIX, `native_file_handle_type` should be `int`. For Windows-based operating systems, it should be `HANDLE`.]

Add the following into Header `<fstream>` synopsis [**fstream.syn**]:

```
template<class charT, class traits>
native_file_handle_type native_file_handle(const basic_filebuf<charT, tr

template<class charT, class traits>
native_file_handle_type native_file_handle(const basic_ifstream<charT, t

template<class charT, class traits>
native_file_handle_type native_file_handle(const basic_ofstream<charT, t

template<class charT, class traits>
native_file_handle_type native_file_handle(const basic_fstream<charT, tr
```

Modify Class template `basic_filebuf` [**filebuf**] § 1

The class `basic_filebuf<charT, traits>` associates both the input sequence and the output sequence with a file. The file has an associated `native_file_handle_type`.

Add the following to the appropriate section of *File-based streams* [**file.streams**]. Replace the * in 29.9.* with the appropriate number.

§ 29.9.* Function template `native_file_handle` [`file.handle`]

```
template<class charT, class traits>
native_file_handle_type native_file_handle(const basic_filebuf<charT, tr
```

Returns: The `native_file_handle_type` associated with the underlying file of `buf`.

```
template<class charT, class traits>
native_file_handle_type native_file_handle(const basic_ifstream<charT, t
template<class charT, class traits>
native_file_handle_type native_file_handle(const basic_ofstream<charT, t
template<class charT, class traits>
native_file_handle_type native_file_handle(const basic_fstream<charT, tr
```

Returns: `native_file_handle(*stream.rdbuf());`

§ 11. Acknowledgements

Thanks to the rest of the co-authors of [\[P1750R0\]](#) for the idea after cutting this functionality out.

A special thanks to Jeff Garland for providing the heads-up about ABI that I totally would've missed.

§ References

§ Informative References

[ACCESS-FILE-DESCRIPTORS]

Bruce S. O. Adams. [file streams and access to the file descriptor](#). URL:

<https://docs.google.com/viewer?>

[a=v&pid=forums&srcid=MTEwODAzNzI2MjM1OTc0MjE3MjkBMDY0OTY1OTUzMjAwNzY0MTA0MjkBakhWMHBFLUNGd0FKATAuMQFpc29jcHAub3JnAXYy&authuser=0](https://docs.google.com/viewer?a=v&pid=forums&srcid=MTEwODAzNzI2MjM1OTc0MjE3MjkBMDY0OTY1OTUzMjAwNzY0MTA0MjkBakhWMHBFLUNGd0FKATAuMQFpc29jcHAub3JnAXYy&authuser=0)

[Boost.IOStreams]

Jonathan Turkanis. [Boost.IOStreams](#). URL:

https://www.boost.org/doc/libs/1_70_0/libs/iostreams/doc/index.html

[N4734]

Jonathan Wakely. [Working Draft, C ++ Extensions for Networking](#). 4 April 2018. URL: <https://wg21.link/n4734>

[P1031R1]

Niall Douglas. [Low level file i/o library](#). URL: <https://wg21.link/p1031r1>

[P1750R0]

Klemens Morgenstern, Jeff Garland, Elias Kosunen, Fatih Bakir. [A Proposal to Add Process Management to the C++ Standard Library](#). URL: <https://wg21.link/p1750r0>

[STD-DISCUSSION-FD-IO]

[File descriptor-backed I/O stream?](#). URL: <https://groups.google.com/a/isocpp.org/forum/#!topic/std-discussion/macDvhFDrjU>

[STD-PROPOSALS-BILLY-ONEAL]

Billy O'Neal. [Comment on 'native_handle for basic_filebuf'](#). URL: <https://groups.google.com/a/isocpp.org/d/msg/std-proposals/oCEErQbI9sM/rMkAMOkxFvMJ>

[STD-PROPOSALS-FD-ACCESS]

[file streams and access to the file descriptor](#). URL: <https://groups.google.com/a/isocpp.org/d/topic/std-proposals/XcQ4FZJKDbM/discussion>

[STD-PROPOSALS-NATIVE-HANDLE]

[native_handle for basic_filebuf](#). URL: <https://groups.google.com/a/isocpp.org/d/topic/std-proposals/oCEErQbI9sM/discussion>

[STD-PROPOSALS-NATIVE-RAW-IO]

[Native raw IO and FILE* wrappers?](#). URL: <https://groups.google.com/a/isocpp.org/d/topic/std-proposals/Q4RdFSZggSE/discussion>